

A Reactive Swarm Controller for the ARGoS Tunnelling Task

Louis Devroye (523920)

Swarm Intelligence, INFO-H-414

Abstract. This report studies a reactive controller for the ARGoS tunnelling project. The implementation combines local repulsion and attraction vectors, a phase-based state machine, and a small set of arena cues to coordinate a swarm of foot-bots without centralized control. Range-and-bearing communication is used to maintain spacing, proximity and light sensing bias motion, and motor-ground readings detect the black target zone. The controller follows a four-state organization: orientation, grouping, tunnel progression, and black-zone motion.

Keywords: swarm robotics · ARGoS · Lua · tunnelling · foot-bot

1 Introduction

The tunnelling task requires a swarm to cross an obstacle-filled arena, reach the target zone on the right, and minimize contamination of that target by obstacles. The score is the number of robots in the target minus the number of obstacles in the target at the end of a run. The project brief further requires multiple runs with different random seeds, boxplots of the final score, and a scalability analysis across swarm sizes.

The controller is written in Lua for the foot-bot platform and uses the local sensing and actuation model described in the course notes for Lua and the foot-bot API [1, 2]. Its design is intentionally reactive: each robot responds to nearby peers, light, floor color, and obstacles rather than relying on a global map or centralized coordination.

As a comparison baseline, the repository also contains a simpler gray-zone escape controller, implemented in the file `gray_escape.lua` (referred to here as the gray-scale baseline). In this baseline, a robot performs a random walk on white floor until a gray or black floor cue is detected; it then stores the escape direction suggested by the proximity field and follows that vector until the floor returns to white. This baseline is intentionally minimal: it does not use the swarm-level grouping or leader-following rules of the main controller, but it provides a direct reference for evaluating how much the main controller improves collective tunnelling performance. The full code is available in the public repository: https://github.com/ldevroye/swarm_intelligence.

This follows the same lesson as Conway’s Game of Life: simple local rules can generate large, surprising, and structured collective behavior.

2 Problem Definition and Requirements

The arena is a rectangular environment with a white starting area on the left, an obstacle field in the middle, and a black target area on the right. The project brief specifies a 1,000 second run, corresponding to 10,000 ARGoS steps, although a shorter runtime may be used if justified in the report.

The available sensors and actuators include proximity, light, motor-ground, range-and-bearing, colored-blob omnidirectional camera, wheels, LEDs, gripper, and turret. The brief emphasizes cooperative behavior, use of swarm robotics principles, repeated runs over 10 seeds, and a scaling study over several swarm sizes. In addition to the final score, the project data file records the number of robots and obstacles in the target at each step.

2.1 Evaluation Criteria

The main quantities to report are:

- final score as a function of swarm size,
- number of robots in the target over time,
- number of obstacles in the target over time,
- variability across the 10 repetitions for each configuration.

3 Controller Overview

The controller is organized as a state machine with four phases: orientation, grouping, tunnel progression, and black-zone motion. The transition logic is driven by local conditions such as the number of neighbors at a target spacing, floor color, and whether the robot has already entered the black target zone.

At a high level, the controller combines three vector sources:

- a Lennard–Jones interaction over range-and-bearing messages,
- a light-seeking vector from the ambient light sensor,
- obstacle repulsion and tangential corrections from proximity readings.

The implementation uses robot LEDs to expose the current phase. The top beacon is green for robots already in the black zone, orange for robots in the tunnel phase, and blue otherwise. Per-robot logs are written to the `logs/` directory.

3.1 Orientation Phase

In the initial phase, robots bias motion toward the light while maintaining distance from nearby robots and obstacles. The controller uses the light vector, the Lennard–Jones interaction, and a small obstacle tangential correction to avoid overcrowding at the start. After a fixed number of steps, the robot moves to the grouping phase.

3.2 Grouping Phase

The grouping phase uses the range-and-bearing system to enforce a loose swarm structure. The code computes a Lennard–Jones interaction at the current peer distance and counts neighbors in a target interval around 80 cm. Once enough neighbors are found consistently for a sufficient number of steps, the robot switches to the tunnel phase.

The controller also computes a leader vector from robots that already advanced into later phases. This creates a simple local bootstrap effect: robots that are already closer to the target influence nearby robots to follow them.

3.3 Tunnel Phase

The tunnel phase reduces the target spacing to 60 cm and strengthens the attraction toward robots already in the tunnel. The phase uses a combination of Lennard–Jones interaction, light attraction, and leader following. If an obstacle is directly ahead, the controller adds a sideways correction so the robot can move around the obstacle field instead of pushing straight through it.

This phase is the main navigation behavior of the current implementation. It is designed to help the swarm advance collectively through the clutter while still preserving a coherent formation.

In that sense, the tunnel phase follows the flocking behavior explored in the exercises: the robots remain locally coupled and keep moving as a group rather than as isolated individuals.

3.4 Black-Zone Phase

When enough floor sensors detect black for several consecutive steps, the robot enters the black-zone state. In this state, the controller keeps the robot in the target region and uses a randomized heading segment to avoid getting stuck on the boundary. If the robot sees the green beacon of another robot already in the black zone, it biases motion toward that robot as a local cue.

The motor-ground readings are central here: they determine when the robot has reached the target and when it should leave the boundary-following motion. This behavior is close to the random-walk exercise: once inside the black zone, the robot follows a randomized walk-like motion with local corrections instead of a precise global plan.

4 Implementation Details

4.1 Sensors and Local Rules

The controller uses the following signals:

- **Proximity sensors:** obstacle repulsion, obstacle tangential motion, and wall/robot blocking.

- **Light sensors:** orientation toward the ambient cue.
- **Motor-ground sensors:** detection of the black target zone.
- **Range-and-bearing:** local communication and Lennard–Jones spacing.
- **Colored-blob camera:** detection of a green beacon emitted by robots already in the black zone.
- **LEDs:** visual phase feedback during simulation.

The wheel command is computed from the target vector angle with a simple proportional steering law. The result is clamped to the allowed wheel speed. This keeps the behavior easy to reason about and consistent with the reactive style encouraged in the project brief.

4.2 State Transitions

The state machine is driven by local counts and thresholds rather than by global knowledge. The most relevant conditions are:

- a minimum number of neighbors in the target spacing interval,
- a minimum duration during which that grouping condition must hold,
- a consecutive black-floor counter before entering the black zone,
- a beacon-based check to bias motion once inside the target region.

In practice, it is difficult to make the different states behave very distinctly, because each phase depends on careful tuning of thresholds and on a growing amount of branching in the control logic. Making each addition exponentially more difficult to maintain.

4.3 Intended Obstacle Handling

The source also contains an obstacle-handling sequence that would be used only in the black zone when a grabbable obstacle is centered in front of the robot and the cooldown timer has expired. If enabled, the robot would first approach the obstacle slowly, accumulate front contact information, close the gripper once alignment is good enough, and switch the turret to passive mode while the object is carried. The robot would then move while holding the obstacle, following a light-directed motion, and release it after the carry phase or if the sequence must be aborted. The sequence is also guarded by checks that stop it if the floor is no longer black or if the controller leaves the black-zone context. Leaving the obstacle in the gray area

5 Experimental Protocol

The project brief requests 10 runs per configuration with different random seeds. The report should present boxplots of the final score for each swarm size, along with time-series plots for the number of robots in the target and the number of obstacles in the target.

5.1 Planned Settings

The main controller is compared against the gray-zone escape baseline, which is implemented in `gray_escape.lua`. The baseline is a reactive reference controller that switches from random exploration on white floor to a directed escape run when gray or black ground is detected. This makes it a useful comparison point for the more structured swarm behavior of the main controller, which relies on local grouping, leader-following, and tunnel-phase transitions.

Table 1. Planned experimental settings. Replace placeholders with the final tested configurations.

Configuration	Value	Notes
Run duration	10,000 steps	Default setting
Seeds per setting	10	Required
Swarm sizes	5, 10, 20, 30	Requested test sizes
Baseline	<code>gray_escape.lua</code>	Gray-scale escape reference controller

The source code for both the main controller and the baseline is available in the public repository at https://github.com/ldevroye/swarm_intelligence.

5.2 Data to Report

Table 2. Summary placeholders for experiment outputs. Replace with computed statistics once the runs are available.

Swarm size	Mean score	Std. dev.	Median score
5	<value>	<value>	<value>
10	<value>	<value>	<value>
20	<value>	<value>	<value>
30	<value>	<value>	<value>

6 Results

The following figures remain to be completed with the final experimental data.

6.1 Discussion Points for the Final Draft

Once the data are inserted, the discussion should answer the following:

- How does performance scale with the number of robots?

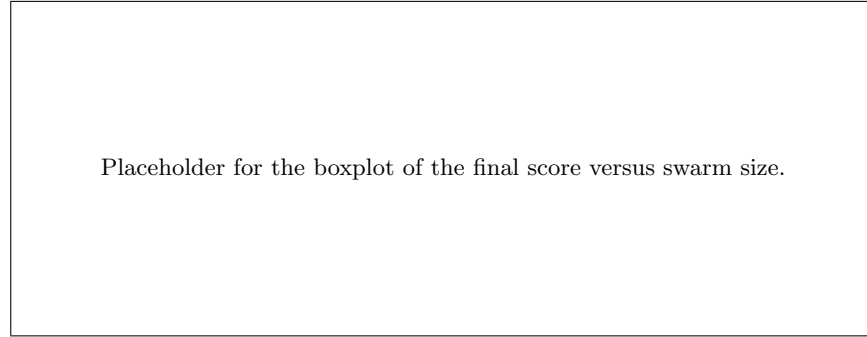


Fig. 1. Final score as a function of swarm size. Each box should summarize 10 repetitions.

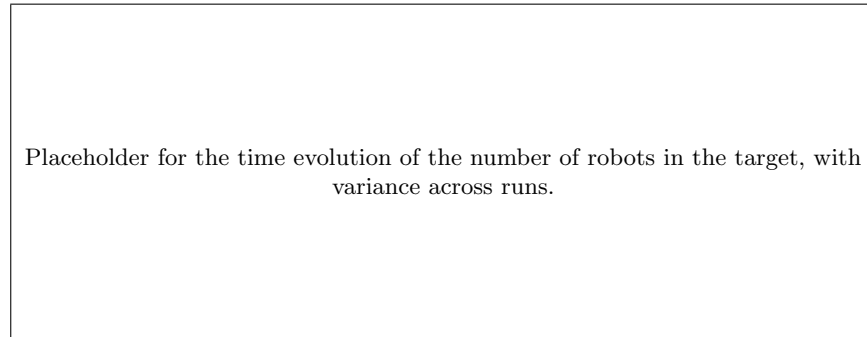


Fig. 2. Robots in the target over time for representative swarm sizes.

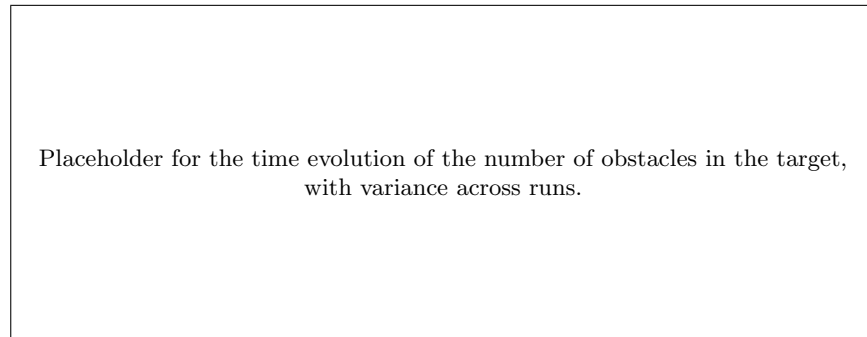


Fig. 3. Obstacles in the target over time for representative swarm sizes.

- Is the performance monotonic, or do some sizes perform worse than nearby sizes?
- Do some swarm sizes exhibit higher variance than others?
- Are the observed trends consistent with the controller’s local rules and phase transitions?

7 Conclusion

The current controller implements a clear reactive structure that is aligned with the swarm robotics principles emphasized in the course: local sensing, distributed coordination, and simple rules that produce collective motion. The report is ready for the final experimental plots and statistics.

References

1. H414 – Swarm Intelligence: Programming the Robots with Lua. Course reference document.
2. H414 – Swarm Intelligence: Sensor and actuator reference for the foot-bot. Course reference document.
3. H414 – Swarm Intelligence: Evaluation and Final Project of INFO-H-414 Swarm Intelligence, Second Session.
4. Springer: Lecture Notes in Computer Science style and sample paper documentation.